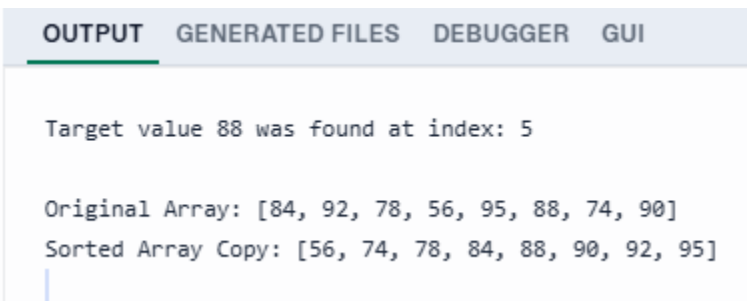```java
import java.util.Arrays;

public class SearchSortMiniTool {

    public static void main(String[] args) {
        int[] originalScores = {84, 92, 78, 56, 95, 88, 74, 90};
        int target = 88;

        int foundIndex = -1;
        for (int i = 0; i < originalScores.length; i++) {
            if (originalScores[i] == target) {
                foundIndex = i;
                break;
            }
        }

        if (foundIndex != -1) {
            System.out.println("Target value " + target + " was found at index: " + foundIndex);
        } else {
            System.out.println("Target value " + target + " was not found in the array.");
        }

        int[] sortedScores = Arrays.copyOf(originalScores, originalScores.length);
        Arrays.sort(sortedScores);

        System.out.println("\nOriginal Array: " + Arrays.toString(originalScores));
        System.out.println("Sorted Array Copy: " + Arrays.toString(sortedScores));

        /*
         * Sorting a copy rather than the original array matters here because the original
         * array indexes preserve data identity (e.g., index 0 corresponds to student 1).
         * Sorting the original array in place breaks this mapping and scrambles the relationship.
         * Keeping a copy preserves the integrity of the original source dataset.
         */
    }
}
```